

Teste assíncrono — Cientista de Dados, maio/2026

Diagnóstico de dados municipais para apoio à inteligência fiscal — caso fictício Vitória/ES

Duração: 72 horas corridas a partir do envio do material por e-mail.

Formato da entrega: arquivos digitais, conforme descrito no item 4, enviados à coordenação do processo seletivo: henrique.cristovao@ufes.br, bem como o compartilhamento do projeto desenvolvido em repositório no GitHub com o *username* = [hmcristovao](#).

Entrevista posterior: após a entrega, o(a) candidato(a) participará de uma entrevista presencial para se apresentar, apresentar a solução desenvolvida e responder a perguntas sobre as decisões tomadas.

1. Contexto do teste

O Lab Cidades desenvolve projetos em várias áreas, entre elas, a articulação de dados municipais, análise territorial, interpretação econômica e apoio à tomada de decisão pública. Neste teste, você receberá uma base fictícia inspirada em problemas de inteligência fiscal municipal, especialmente relacionados a IPTU e ISS.

Os registros simulam débitos municipais, dados cadastrais, informações territoriais, canais de contato e indicadores econômicos de contexto. A base foi construída propositalmente com problemas de qualidade para que você possa demonstrar capacidade de leitura crítica, preparação, análise e comunicação de dados.

Os dados são totalmente fictícios e não representam contribuintes, imóveis, empresas ou valores reais do município de Vitória/ES.

2. Arquivos fornecidos

Arquivo	Descrição
<code>enunciado_teste_cientista_dados.pdf</code>	Enunciado do teste com as orientações detalhadas.
<code>debitos_municipais_ficticios_vitoria_es.csv</code>	Tabela com 80.000 registros fictícios de débitos municipais de Vitória/ES.

<code>bairros_referencia.csv</code>	Tabela de referência contendo informações sobre os bairros.
<code>dicionario_dados.pdf</code>	Dicionário de dados sobre as variáveis da tabela.

3. Ferramentas permitidas

Use quaisquer ferramentas computacionais que julgar necessário. A parte do desenvolvimento do programa, obrigatoriamente deve ser feita com a linguagem Python.

O uso de IA generativa é permitido, mas deve ser declarado. O importante é demonstrar capacidade de validar criticamente as respostas da ferramenta, tomar decisões justificadas e explicar o que foi feito.

4. O que deve ser produzido

1. Uma versão tratada da base principal, em formato CSV.
2. Um relatório com todos os resultados e com um memorial metodológico explicando as decisões de limpeza, padronização, cálculo e interpretação, e demais procedimentos realizados. O documento do relatório deve estar em formato PDF.
3. Projeto desenvolvido em repositório no GitHub, compartilhado com *username* = [hmcristovao](#), contendo instruções de uso no readme.

Obs.: Não se espera o esgotamento de todas as possibilidades analíticas. Será valorizada a capacidade de priorizar problemas relevantes, justificar decisões e produzir uma solução correta, reprodutível e bem documentada.

5. Tarefas obrigatórias

5.1 Diagnóstico da qualidade dos dados

- Identifique problemas de campos ausentes, duplicidades, inconsistências de bairro/regional/município, formatos monetários, datas inválidas, categorias não padronizadas e inconsistências entre telefone, e-mail e canal de contato.
- Indique quais problemas você priorizou corrigir e quais decidiu apenas registrar como limitação.

5.2 Preparação e padronização mínima

- Padronize, na medida do possível, nomes de bairros e regionais usando a tabela `bairros_referencia.csv`.
- Padronize variáveis relevantes, incluindo valores monetários, situação do pagamento, tipo de tributo, setor econômico, porte estimado, canal de contato, entre outras.
- Trate duplicidades de forma explícita, justificando manter, remover ou consolidar registros.

5.3 Indicadores descritivos

- Calcule indicadores como valor total da dívida, número de registros inadimplentes, valor médio e mediano da dívida, distribuição por faixa de valor, concentração da dívida, registros com contato válido e distribuição por bairro/regional.
- Separe, quando fizer sentido, IPTU e ISS, pois eles têm lógicas econômicas e cadastrais diferentes.
- Apresente um histograma da distribuição dos valores de débito (`valorDebitoOriginal`), considerando os registros classificados como inadimplentes, conforme sua padronização. Caso necessário, utilize escala logarítmica, recorte de outliers ou faixas de valor, justificando a escolha.
- Investigue uma possível correlação, na tabela `bairros_referencia.csv`, entre as variáveis:
 - `indicadorVulnerabilidadeBairro`
 - `rendaMediaBairroEstimada`

Use como apoio um diagrama de dispersão. Faça as interpretações necessárias e conclua sobre a possível correlação e o significado dela.

5.4 Programa desenvolvido em Python

- Produzir um programa em Python que leia a base principal, já devidamente tratada, e um arquivo de parametrização em formato `.txt`; aplique o filtro indicado para `situacaoPagamento` (ver exemplo abaixo); selecione as variáveis listadas no arquivo de parametrização, na ordem informada; e gere um terceiro arquivo CSV de saída contendo apenas os registros e campos solicitados.
- O arquivo de configuração que será fornecido possui conteúdo variável fornecido pelo usuário. Mas, ele segue estritamente o seguinte formato:
 - A primeira linha indica o filtro a ser realizado para a situação do pagamento, que pode ser 'Em aberto', 'Atrasado', 'Dívida ativa', 'Pago', 'Parcelado' ou 'Quitado', conforme sintaxe mostrada no exemplo abaixo.

- A segunda linha sempre fica em branco.
 - A terceira linha contém o termo 'Variáveis:'
 - A quarta linha e as demais linhas contém as variáveis que devem aparecer no arquivo final, na ordem especificada, e com os seus respectivos valores.
- Exemplo de arquivo de parametrização:

Nome do arquivo: `parametros.txt`

```
situacaoPagamento = 'Dívida Ativa'

Variáveis:
idContribuinte
tipoContribuinte
anoDebito
bairro
tipoTributo
```

- Na primeira linha no arquivo de parametrização, o algoritmo deve aceitar variações de acentuação e capitalização, como “Dívida Ativa”, “dívida ativa” e “DÍVIDA ATIVA”, desde que correspondam a uma categoria válida após padronização.
- O programa deve validar se:
 - o arquivo de entrada existe;
 - o arquivo de parâmetros está no formato esperado;
 - a situação de pagamento informada é válida;
 - todas as variáveis solicitadas existem na base;
 - o arquivo de saída foi gerado corretamente;
 - erros sejam comunicados de forma compreensível ao usuário.
- O projeto deve conter instruções claras de execução no arquivo README.md. Você poderá implementar a execução de diferentes formas, desde que seja possível reproduzir o processamento a partir da base tratada e do arquivo de parâmetros fornecido.
- Deve-se informar no relatório as bibliotecas utilizadas e a lógica geral do processamento.

6. Registro obrigatório de uso de IA

Inclua uma seção intitulada “Registro de uso de IA”. Ela deve conter:

- Ferramentas utilizadas;
- Em quais etapas foram utilizadas;

- Exemplos de comandos ou prompts relevantes;
- O que foi aceito, rejeitado ou corrigido;
- Como você verificou se as respostas estavam corretas.

7. Entrevista posterior

Na entrevista, você deverá se apresentar brevemente, explicar a solução desenvolvida e responder a perguntas sobre as soluções desenvolvidas. Poderá ser pedido para você explicar uma transformação realizada, reinterpretar um gráfico, justificar um critério de priorização, explicar partes do código desenvolvido ou identificar limitações do seu próprio trabalho.

8. Critérios gerais de avaliação

- Compreensão do problema e aderência ao contexto municipal.
- Diagnóstico e tratamento da qualidade dos dados.
- Indicadores, tabelas e gráficos.
- Corretude, eficácia e documentação do programa desenvolvido.
- Clareza da comunicação e organização da entrega.
- Transparência no uso de IA e capacidade de validação crítica.